JavaScript Timers

setTimeout · clearTimeout · setInterval · clearInterval

Revision notes: concepts, my mistakes, and practice

Contents

PART 1 · Concepts 2

1.1 What is a timer? 2

1.2 setTimeout: run ONCE after a delay 2

1.3 clearTimeout: cancel a timeout BEFORE it fires 2

1.4 setInterval: run REPEATEDLY every N ms 3

1.5 clearInterval: stop the repeating timer 3

1.6 The matching rule 3

1.7 WHEN does code run? (most important) 3

1.8 Order INSIDE the callback matters 4

1.9 Execution order with multiple timers 4

1.10 Useful helpers learned along the way 5

1.11 Real-world scenarios 5

1.12 In React 6

PART 2 · My Understanding and Mistakes 6

Exercise 1 · Welcome message 6

Doubt · “cancel the timer before it fires” 6

Exercise 2 · Undo delete 7

Exercise 3 · Exam timer 7

Exercise 4 · Loading dots 8

Exercise 5 · Bug hunt 9

PART 3 · My Recurring Patterns 10

PART 4 · Pending Practice 10

How to read the markers

✓ Correct / good practice	✗ Wrong / mistake	⚠ Works, but needs fixing
Black label = a RULE to memorise	**Thin label** = tip, analogy, lesson	Code blocks: **bold** = keyword, *italic* = comment

PART 1 · Concepts

1.1 What is a timer?

A timer is a request to the browser or Node.js: *“Call this function later.”*

JavaScript does **not** wait for the timer. It schedules it and moves straight to the next line.

1.2 setTimeout: run ONCE after a delay

```
const id = setTimeout(() => {
  console.log("Runs once, after 2 seconds");
}, 2000);
```

- Runs the callback **one time**.
- The delay is a *minimum*, not an exact time.
- Returns an **id** (a ticket number) that you can use to cancel it.

1.3 clearTimeout: cancel a timeout BEFORE it fires

MEANING

“Fires” = the timer finishes and runs your function.

Alarm analogy

Phone alarm	JavaScript
Set an alarm for 7:00	`setTimeout(fn, delay)`
Alarm number / name	`id`
Turn it off at 6:50, so it never rings	`clearTimeout(id)` before it fires ✓
Try to turn it off at 7:05, too late	`clearTimeout(id)` after it already fired: nothing to cancel

```
const alarmId = setTimeout(() => console.log("RING RING!"), 3000);

setTimeout(() => {
  clearTimeout(alarmId);
  console.log("Alarm cancelled. It will NOT ring.");
}, 1000);
```

1.4 setInterval: run REPEATEDLY every N ms

```
const id = setInterval(() => {
  console.log("Runs every 1 second");
}, 1000);
```

It runs **forever** until you stop it.

1.5 clearInterval: stop the repeating timer

```
clearInterval(id);
```

If you forget to clear an interval, it keeps running in the background. That causes memory leaks, duplicate API calls, and errors after the user leaves the page.

1.6 The matching rule

Created with	Cancel with
`setTimeout`	`clearTimeout`
`setInterval`	`clearInterval`

RULE

Always match the pair. Browsers and Node.js happen to accept either one, but mixing them confuses readers and gets flagged in code review.

1.7 WHEN does code run? (most important)

```
let count = 0;

const id = setInterval(() => {
  // code INSIDE the callback runs EVERY tick
  count++;
}, 1000);

// code OUTSIDE the callback runs ONCE, immediately (at ~0 ms)
console.log(count); // 0
```

Where the code is	How often it runs
Outside the callback	**Once**, at the start, before any tick
Inside the callback	**Every tick**

RULE

If something must be checked repeatedly (like “is count 3 yet?”), the check must live **inside** the callback.

1.8 Order INSIDE the callback matters

Lines inside the callback run top to bottom. `clearInterval(id)` stops **future** ticks. It does **not** stop the rest of the **current** tick.

✓ CHECK AFTER THE VALUE CHANGES: PRINTS “HI” 3 TIMES

```
let count = 0;
const id = setInterval(() => {
  console.log("Hi");
  count++;
  if (count === 3) {
    clearInterval(id);
  }
}, 1000);
```

⚠ CHECK BEFORE THE VALUE CHANGES: PRINTS “HI” 4 TIMES

```
let count = 0;
const id = setInterval(() => {
  if (count === 3) {
    clearInterval(id); // stops FUTURE ticks only...
  }
  console.log("Hi");   // ...this line still runs in the current tick!
  count++;
}, 1000);
```

Fix for the second version: add `return;` right after `clearInterval(id);` to exit the callback immediately.

ANALOGY: WASHING MACHINE

Pressing stop lets the **current** wash finish; only the **next** washes are cancelled.

1.9 Execution order with multiple timers

```
setTimeout(() => console.log("A"), 0);
console.log("B");
setTimeout(() => console.log("C"), 100);
setTimeout(() => console.log("D"), 50);
console.log("E");
// Output: B, E, A, D, C
```

1. Normal (synchronous) code runs first: **B, E**.
2. Then timers fire in order of their delay: **A** (0), **D** (50), **C** (100).

Even a `0 ms` timer waits until the current code has finished.

1.10 Useful helpers learned along the way

Tool	What it does	Example
`%` (remainder)	Makes a value **cycle**	`count % 3` gives 0, 1, 2, 0, 1, 2…
`.repeat(n)`	Repeats a string	`".".repeat(3)` gives `"..."`
`===`	Strict equality (value **and** type)	`"6" === 6` is `false`
`const` vs `let`	`const` = never reassigned; `let` = changes	`const id` , `let count`

The % cycle (“Loading.” → “Loading...” repeating)

count	count % 3	+ 1 = dots
0	0	1
1	1	2
2	2	3
3	0	1 (cycle restarts)

1.11 Real-world scenarios

Scenario	Timer	Cancel when
Toast “Saved ✓” auto-hide	`setTimeout`	The user closes it early
Email “Undo” delete	`setTimeout`	The user clicks Undo (`clearTimeout`)
Search box debounce	`setTimeout`	Each new keystroke resets it
OTP “Resend in 30s”	`setInterval`	It reaches 0
Live clock / exam timer	`setInterval`	Time over or the component unmounts
Polling job status	`setInterval`	Status = done **or** max tries reached
Loading dots / carousel	`setInterval` + `%`	Content loaded

DEBOUNCE PATTERN

```
let timerId;
function search(text) {
  clearTimeout(timerId);            // cancel the previous pending call
  timerId = setTimeout(() => {
    console.log("API call with:", text);
  }, 300);
}
```

1.12 In React

Start the timer inside `useEffect` and **always** clear it in the cleanup function:

```
useEffect(() => {
  const id = setInterval(() => setTime(new Date()), 1000);
  return () => clearInterval(id); // runs when the component unmounts
}, []);
```

PART 2 · My Understanding and Mistakes

Exercise 1 · Welcome message

✓ CORRECT

MY CODE

```
console.log("Page loaded")

setTimeout(()=>{
    console.log("Welcome, Selva!");
},2000)
```

✓ GOT RIGHT	⚠ STYLE ISSUES
`setTimeout` runs once after a delay	Semicolons were inconsistent (one line had one, others didn't)
	No spaces in `()=>{` and `},2000)`

Doubt · "cancel the timer before it fires"

I didn't understand what *"before it fires"* meant.

CLEARED WITH: THE ALARM ANALOGY

"Fires" = the timer finishes and runs the function.

Cancelling only works while the timer is **still waiting**.

Exercise 2 · Undo delete

⚠ WORKED, BUT WRONG METHOD

MY CODE

```
let id = setTimeout(()=>{
    console.log("Email permanently deleted");
},5000);

setTimeout(()=>{
    clearInterval(id);   // <-- WRONG: should be clearTimeout
    console.log("Undo successful, email restored")
},2000)
```

✗ Mistake	✓ Correct
Cancelled a `setTimeout` with `clearInterval`	`setTimeout` pairs with `clearTimeout`
`let id`	`const id` (it's never reassigned)

LESSON

"It works" ≠ "it's correct". Match the pair.

Exercise 3 · Exam timer

✓ CORRECT

MY CODE

```
let value = 0;

const id = setInterval(()=>{
    value++;
    if(value==6){
        clearInterval(id);
        console.log("Time over! Submit your paper")
    }
    else{
    console.log(`${value} second passed`)
    }
},1000)
```

✓ IMPROVED FROM FEEDBACK	⚠ STILL TO FIX
Used `const` for `id` (applied feedback from Ex 2)	`==` → use `===`
Used `let` for `value` (it changes)	“2 second passed” → “2 **seconds** passed” (plural)
Correct pair: `setInterval` + `clearInterval`	Indentation inside `else`; missing semicolons
Put the `if` check **inside** the callback	

Exercise 4 · Loading dots

Attempt 1

✗ WRONG: THE PATTERN DIDN’T REPEAT

MY CODE (THE LOOP)

```
for(let i=0;i<n;i++){ temp+="." }
// count goes 0, 1, 2, 3, 4... and never resets
```

The output kept growing: `Loading.` → `Loading..` → `Loading...` → `Loading....` → …

✗ Mistake	✓ Fix
`count` never cycles back	Use `% 3` to make it repeat 0, 1, 2, 0, 1, 2…

Attempt 2

✓ CORRECT

MY CODE (THE LOOP)

```
for (let i = 0; i < (n % 3); i++) {
  temp += ".";
}
// "Loading." + printDots(count)  ->  1 fixed dot + 0-2 extra = 1-3 dots
```

✓ GOOD	⚠ STILL TO FIX
Used `%` correctly	`let id` → `const id` (**missed a second time**)
Kept my own helper function, a valid approach	Missing semicolons
The fixed `"."` cleverly replaces the `+ 1`	Add a comment explaining 1 fixed dot + 0–2 extra

Exercise 5 · Bug hunt

THE BUGGY CODE

```
let count = 0;
const id = setInterval(() => {
  console.log("Hi");
  count++;
}, 1000);

if (count === 3) {
  clearInterval(id);
}
```

My first answer

✗ WRONG

"id should be initialised first": `id` is already initialised on line 2 (`const id = setInterval(...)`).

The real reason

Time	What happens
0 ms	`setInterval` is **scheduled** (not run)
0 ms	The `if` runs **immediately**: `count` is 0, so it's false and nothing is cleared
0 ms	The file ends, and **the** `if` **never runs again**
1 s, 2 s, 3 s…	"Hi" forever, because nobody checks `count` anymore

My second answer

✓ CORRECT

"The whole if block should be inside the setInterval function."

Then I needed help with

Whether the `if` goes before or after `count++` (see section 1.8):

- **After** `count++` : 3 "Hi" ✓
- **Before** `count++` : 4 "Hi", because `clearInterval` doesn't stop the current tick ⚠

PART 3 · My Recurring Patterns

Pattern	Times	Fix
`let` used for a timer `id` that never changes	2 (Ex 2, Ex 4)	Timer ids are **always** `const`
Missing or inconsistent semicolons	Every exercise	Turn on Prettier + "Format on Save" in VS Code
No spaces around `=>`, `{`, commas	Ex 1–4	Prettier fixes this automatically
`==` instead of `===`	Ex 3	Always use `===`
Confusion about **when** code runs (once vs every tick)	Ex 5	Outside the callback = once; inside = every tick

MY MAIN CONCEPTUAL GAP: TIMING

Code **outside** a timer callback runs **once**, immediately.

Code **inside** runs on **every tick**.

Checks that must repeat go **inside**, right **after** the value changes.

PART 4 · Pending Practice

Tick each box when done. Use the lines to write your prediction or answer.

☐ **Ex 5:** write the fixed code myself (no copy-paste) and confirm exactly 3 "Hi".

☐ **Ex 5 Option B:** put the `if` before `count++` and count the "Hi"s. Then fix it with `return`.

☐ **Ex 1 question:** add `console.log("Bye");` at the end. Does "Welcome" or "Bye" print second?

☐ **Ex 2 experiment:** change the undo time `2000` → `6000`. What prints, and in what order? (the "Undo clicked too late" edge case)

- [ ] **Ex 3 question:** if the `if` uses `value > 6` instead of `value === 6`, when does the timer stop?

- [ ] **Ex 4 experiment:** make the dots cycle 1 → 2 → 3 → 4 → 1. Which single number changes?

- [ ] Practise `timers-practice.js` (exercises 1–10) and compare with `timers-solutions.js`.

ONE-LINE SUMMARY

`setTimeout` = do it later, once · `setInterval` = keep doing it every N ms

`clearTimeout` / `clearInterval` = stop it (match the pair!)

Always clear what you start.